

Практическая работа №3

Теоретическое введение

Ansible и Docker — это два мощных инструмента, которые могут значительно упростить развертывание, управление и автоматизацию приложений и инфраструктуры в целом. Давайте рассмотрим небольшое теоретическое введение по использованию Ansible вместе с Docker.

Ansible — это инструмент для автоматизации конфигурации и управления компьютерными системами. Он основан на модели управления конфигурацией (Configuration Management) и использует YAML для описания задач (playbooks), которые определяют состояние системы и действия, необходимые для достижения этого состояния. Ansible может управлять множеством узлов одновременно и имеет механизмы для управления переменными, шифрования данных, обработки ошибок и многое другое.

Docker — это платформа для разработки, доставки и запуска приложений в контейнерах. Контейнеры обеспечивают легковесное и изолированное окружение для приложений, что позволяет запускать приложения в любом окружении без изменения кода. Docker позволяет эффективно управлять зависимостями приложений, обеспечивает единое окружение для разработки и развертывания, и облегчает масштабирование и управление приложениями.

Использование Ansible с Docker позволяет автоматизировать процессы развертывания и управления контейнерами Docker. Например, Ansible может использоваться для следующих задач:

Установка и настройка Docker: Ansible может автоматически устанавливать и настраивать Docker на целевых узлах.

Управление контейнерами Docker: Ansible может управлять жизненным циклом контейнеров Docker, включая создание, запуск, остановку и удаление контейнеров.

Конфигурирование и обновление контейнеров: Ansible может автоматически настраивать и обновлять контейнеры Docker, включая установку и настройку необходимых зависимостей и обновление приложений внутри контейнеров.

Оркестрация контейнеров: Ansible может использоваться для управления множеством контейнеров и оркестрации их работы в различных средах.

Интеграция с другими инструментами и системами: Ansible интегрируется с множеством других инструментов и систем, что позволяет создавать сложные и мощные автоматизированные рабочие процессы.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>
4. Официальный сайт Ansible: <https://www.ansible.com/>

Задание

Практическое задание: Установка Ansible и использование его с Docker

Цели:

- Установить Ansible.
- Создать плейбук Ansible, который бы проверял наличие Docker в системе, и, в случае его отсутствия Docker, устанавливал бы его.
- Использовать этот плейбук для установки Docker.
- Создать плейбук который выполняет установку Docker и запускает любые два контейнера.
- Использовать созданный плейбук.

Установка Ansible

Установить Ansible можно с помощью команды:

```
sudo dnf install ansible
```

Использование Ansible и Создание плейбуков

Плейбук в Ansible представляет собой текстовый файл в формате YAML, который содержит набор инструкций (задач) для автоматизации определенных действий на целевых хостах. Плейбук определяет, какие задачи должны быть выполнены на целевых хостах и в каком порядке.

Каждая задача в плейбуке состоит из названия, модуля Ansible, параметров модуля и, возможно, дополнительных опций. Модули Ansible представляют собой инструменты, которые выполняют определенные действия на целевых хостах, такие как управление пакетами, работа с файлами, выполнение команд и т. д.

Плейбуки позволяют организовать автоматизированные рабочие процессы, такие как установка и настройка программного обеспечения, обновление системы, настройка сервисов, развертывание приложений и многое другое.

Структура плейбука Ansible представляет собой организацию задач и настроек, необходимых для автоматизации определенных действий на целевых хостах. Общая структура плейбука выглядит следующим образом:

```
---
- name: Название плейбука
  hosts: группа_хостов
  become: да/нет
  become_user: пользователь
  become_method: sudo/su/pbrun/doas
  gather_facts: да/нет
  vars:
    переменная1: значение1
    переменная2: значение2
  vars_files:
    - путь_к_файлу_с_переменными
  tasks:
    - name: Название задачи
      модуль: параметры
      loop: "{{ список }}"
      when: условие
      notify: название_обработчика
      tags:
        - тег1
        - тег2
  handlers:
    - name: Название обработчика
      модуль: параметры
```

name: описание плейбука.

hosts: группа хостов, на которых будут выполняться задачи.

`become`: указывает, нужно ли становиться суперпользователем (yes) или нет (no).

`become_user`: имя пользователя, от имени которого выполняются задачи (по умолчанию root).

`become_method`: метод, используемый для выполнения команд от имени суперпользователя (по умолчанию sudo).

`gather_facts`: флаг, определяющий, нужно ли собирать факты о хостах перед выполнением задач (по умолчанию yes).

`vars`: переменные, используемые в плейбуке.

`vars_files`: файлы, содержащие дополнительные переменные для плейбука.

`tasks`: список задач, которые необходимо выполнить на целевых хостах.

`handlers`: обработчики, которые запускаются при выполнении определенных условий.

`notify`: название обработчика, который будет вызван при успешном выполнении задачи.

`tags`: теги, используемые для группировки задач и их выборочного выполнения.

Это основная структура плейбука Ansible, которая позволяет организовать автоматизацию различных действий на целевых хостах с помощью простого и понятного синтаксиса YAML.

Файл `hosts` в Ansible представляет собой текстовый файл, в котором определяются целевые хосты (серверы или устройства), на которых Ansible будет выполнять задачи. Этот файл используется для организации инвентаря, то есть списка всех управляемых хостов, и определения их свойств, таких как IP-адреса, порты, группировки и переменные.

Файл `hosts` обычно располагается в директории проекта Ansible или в определенном расположении, указанном в конфигурационном файле Ansible (`ansible.cfg`). Формат файла `hosts` довольно простой и имеет следующую структуру:

```
[group1]
host1      ansible_host=IP_address      ansible_port=22
ansible_user=user
ansible_ssh_private_key_file=/path/to/private/key

[group2]
host2      ansible_host=IP_address      ansible_port=22
ansible_user=user
ansible_ssh_private_key_file=/path/to/private/key

[group3:children]
group1
group2
```

Где:

group1, group2, group3 - это группы хостов.

host1, host2 - это имена хостов.

ansible_host - IP-адрес хоста.

ansible_port - порт SSH на хосте (по умолчанию 22).

ansible_user - имя пользователя для подключения к хосту.

ansible_ssh_private_key_file - путь к файлу с приватным ключом SSH.

Файл hosts позволяет организовывать хосты в группы для удобства управления ими и применения к ним различных настроек и действий. Кроме того, он также поддерживает динамическую инвентаризацию, которая позволяет автоматически обнаруживать и добавлять новые хосты в инвентарь.

После создания плейбука и файла hosts необходимо запустить плейбук.

Сделать это можно при помощи команды:

```
ansible-playbook -i hosts <Ваш_плейбук>.yaml
```

Флаг `-i` позволяет указывать файл `hosts` из директории, в которой вы создали ваш плейбук.

Вот пример рабочего плейбука и файла `hosts`, который позволяет установить Python на локальной машине:

Плейбук:

```
- hosts: all
  gather_facts: False

  tasks:
    - name: install python 3
      raw: test -e /usr/bin/python || (dnf -y update &&
dnf install -y python-minimal)
```

Файл `hosts`:

```
[local]
localhost ansible_connection=local
```

Вопросы к практической работе

1. Каковы основные преимущества использования Ansible для автоматизации операций в среде Docker?
2. Как создать и настроить файлы плейбуков Ansible для управления контейнерами Docker?
3. Как использовать модуль `'docker_container'` в Ansible для управления жизненным циклом контейнеров Docker?
4. Как настроить инвентарь в Ansible для управления Docker-хостами и контейнерами?
5. Каким образом можно управлять сетевыми настройками контейнеров Docker с помощью Ansible?

6. Как выполнить установку Docker Engine на удаленных хостах с использованием Ansible?
7. Как настроить Docker Compose с помощью Ansible для развертывания многоконтейнерных приложений?
8. Как использовать Ansible Vault для безопасного хранения конфиденциальных данных, таких как пароли или ключи, при управлении Docker-контейнерами?
9. Как использовать роли Ansible для организации и повторного использования конфигурации Docker-контейнеров?
10. Каковы советы по безопасности при работе с Ansible и Docker, чтобы обеспечить надежную и безопасную автоматизацию инфраструктуры?

Критерии оценки

- Установлен Ansible.
- Создан плейбук который бы проверял наличие Docker в системе, и, в случае его отсутствия Docker, устанавливал бы его.
- Создан файл hosts.
- Создан плейбук который выполняет установку Docker и запускает любые два контейнера.
- Проверена работоспособность созданных плейбуков.
- Составлен отчет о проделанной работе со скриншотами её выполнения.
- Даны ответы на вопросы к практической работе.